

Deep Learning with Neural Networks

Unsupervised Deep Learning (I): autoencoders

Alejandro Lancho Serrano, alancho@ing.uc3m.es

Material original por:
Pablo Martínez Olmos,
José Carlos Aradillas,
Ignacio Peis

Unsupervised vs Supervised

Supervised learning

Data: (x, y)

x is data, y is label

Goal: Learn function to map
 $x \rightarrow y$

Examples: Classification, regression, object detection, semantic segmentation, etc.

Unsupervised learning

Data: x

x is data, no labels!

Goal: Learn the *hidden* or *underlying structure* of the data

Examples: Clustering, feature or dimensionality reduction, etc.

Unsupervised Learning

Non-probabilistic Models

- Sparse Coding
- Autoencoders
- Others (e.g. k-means)

Probabilistic (Generative) Models

Tractable Models

- Fully observed Belief Nets
- NADE
- PixelRNN

Non-Tractable Models

- Boltzmann Machines
- Variational Autoencoders
- Helmholtz Machines
- Many others...

- Generative Adversarial Networks
- Moment Matching Networks
- Diffusion models

Explicit Density $p(x)$

Implicit Density

Unsupervised Learning

Non-probabilistic Models

- Sparse Coding
- Autoencoders
- Others (e.g. k-means)

Probabilistic (Generative) Models

Tractable Models

- Fully observed Belief Nets
- NADE
- PixelRNN

Non-Tractable Models

- Boltzmann Machines
- Variational Autoencoders
- Helmholtz Machines
- Many others...

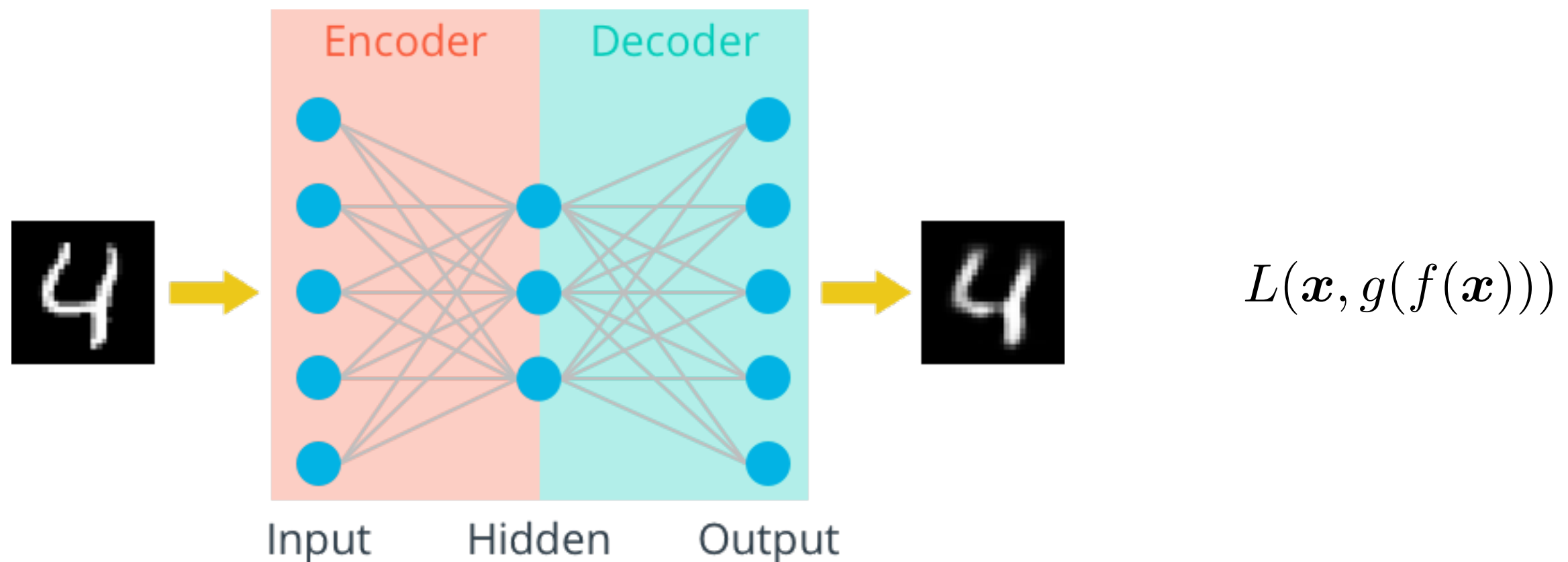
- Generative Adversarial Networks
- Moment Matching Networks
- Diffusion models

Explicit Density $p(x)$

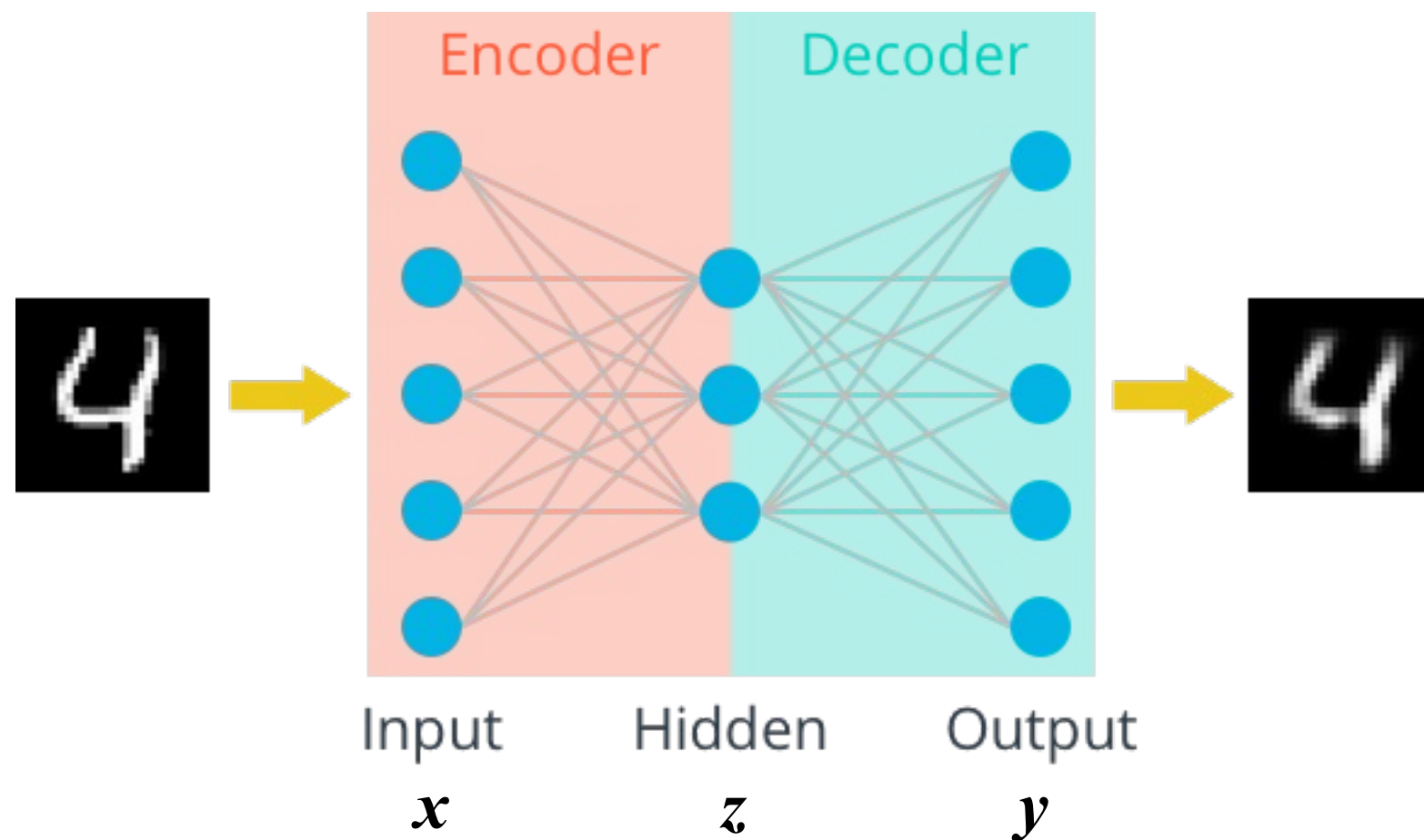
Implicit Density

Deep Autoencoder

- An autoencoder is a neural network that is trained to attempt to copy its input to its output
- Internally, it has a low-dimensional hidden representation
- Two parts: encoder and decoder
- Autoencoders have been traditionally used for dimensionality reduction or feature learning



Deep Autoencoder



- The “encoder” learns mapping from the data, $\mathbf{x}$, to a low-dimensional latent space, $\mathbf{z}$

$$\mathbf{z} = f(\mathbf{x})$$

- The “decoder” learns mapping back from latent space, $\mathbf{z}$, to a reconstructed observation, $\mathbf{y}$

$$\mathbf{y} = g(\mathbf{z}) = g(f(\mathbf{x}))$$

- The loss function doesn't use any labels

$$L(\mathbf{x}, g(f(\mathbf{x})))$$

Dimensionality of latent space → reconstruction quality

Autoencoding is a form a compression

2D latent space



5D latent space



Ground Truth



Autoencoders for representation learning

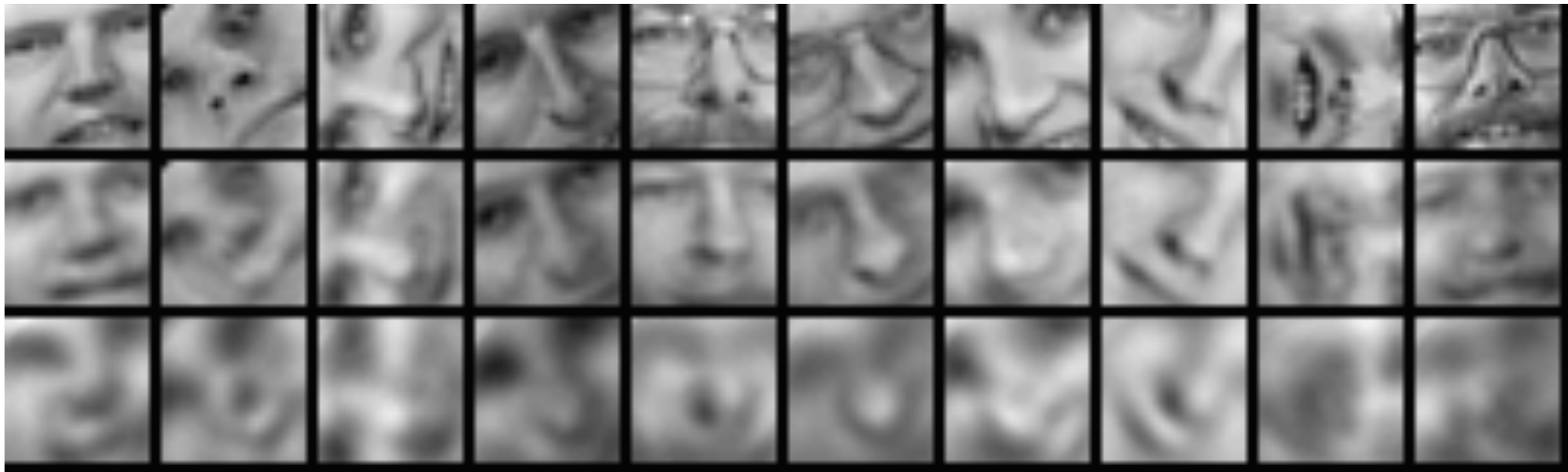
Bottleneck hidden layer forces network to learn a compressed latent representation

Reconstruction loss forces the latent representation to capture (or encode) as much “information” about the data as possible

Autoencoding = **Automatically encoding** data; “Auto” = **self**-encoding

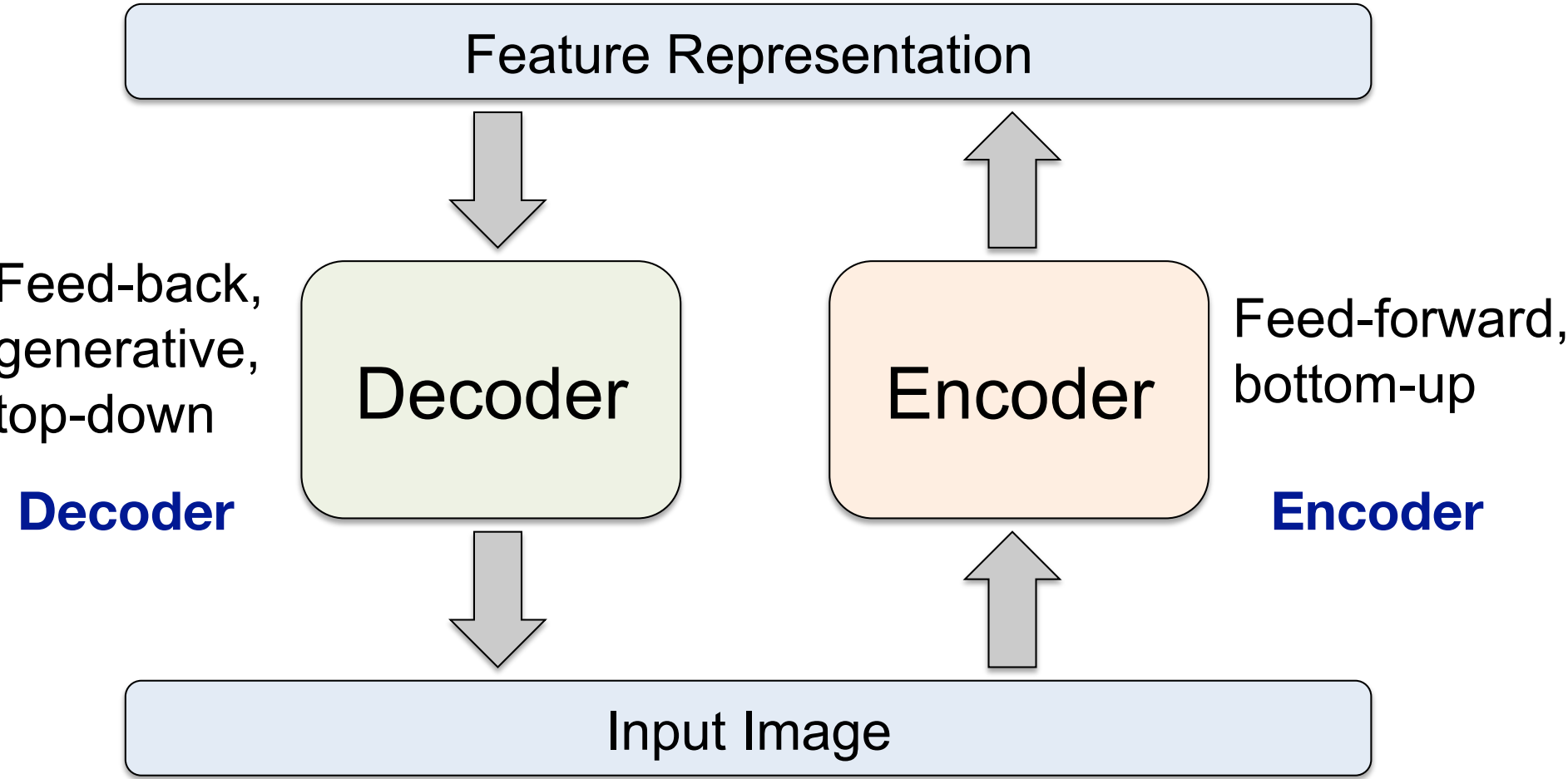
Deep Autoencoder

- 25x25 – 2000 – 1000 – 500 – 30 autoencoder to extract 30-D real-valued codes for Olivetti face patches.

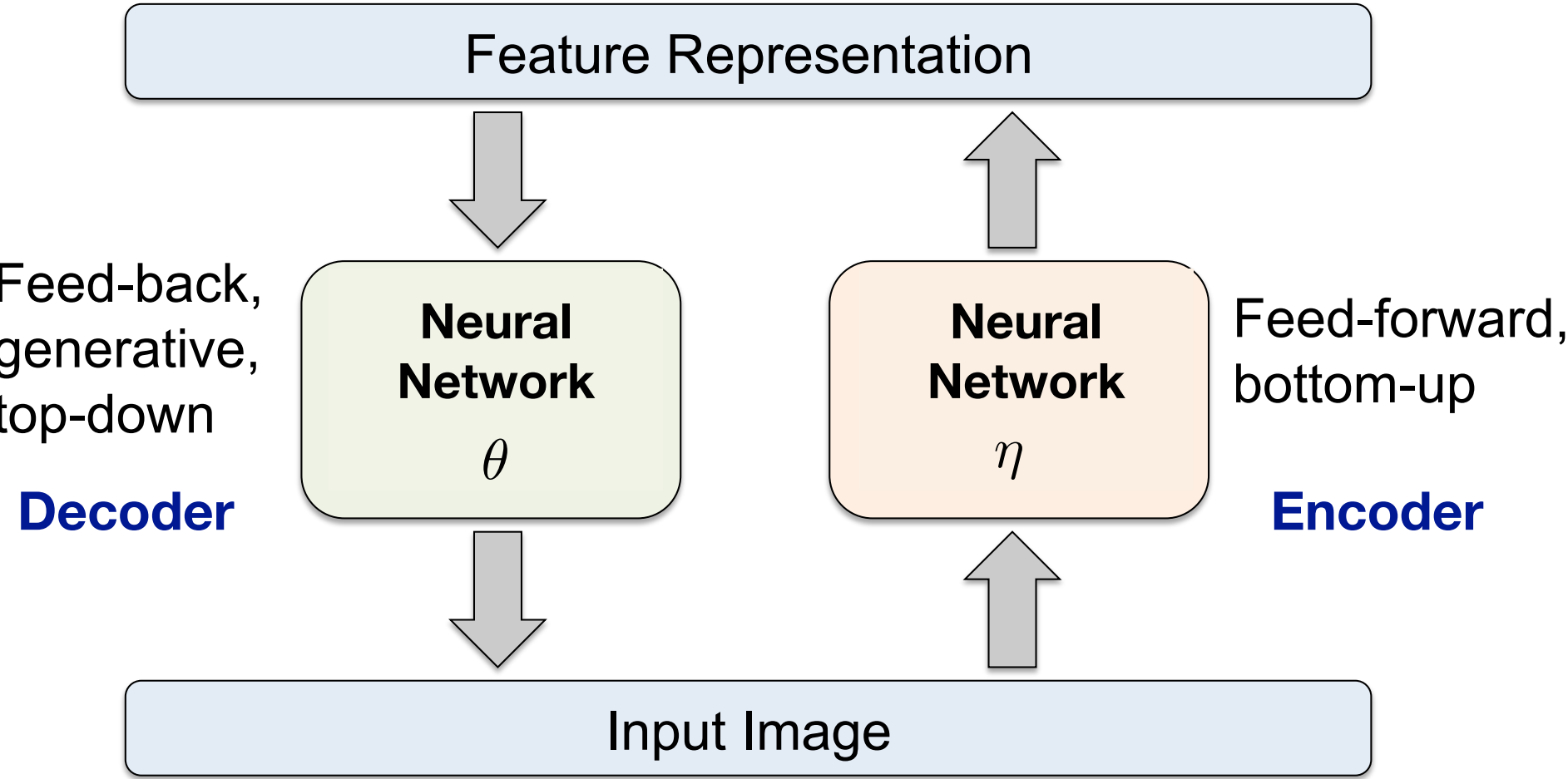


- **Top:** Random samples from the test dataset.
- **Middle:** Reconstructions by the 30-dimensional deep autoencoder.
- **Bottom:** Reconstructions by the 30-dimensional PCA.

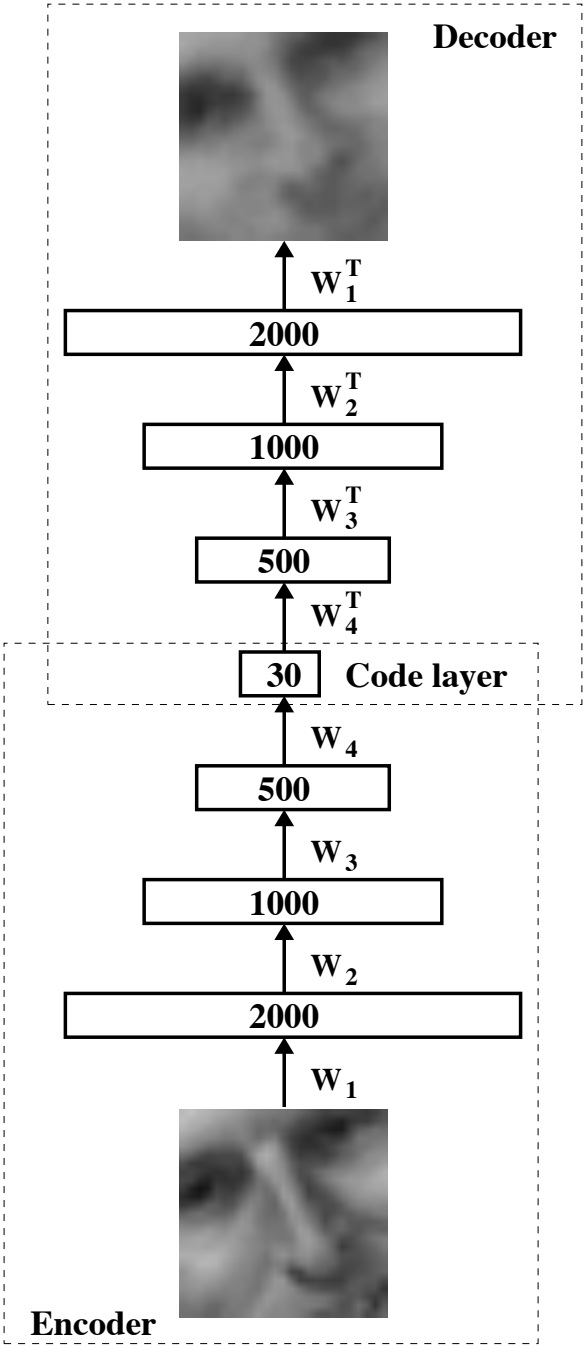
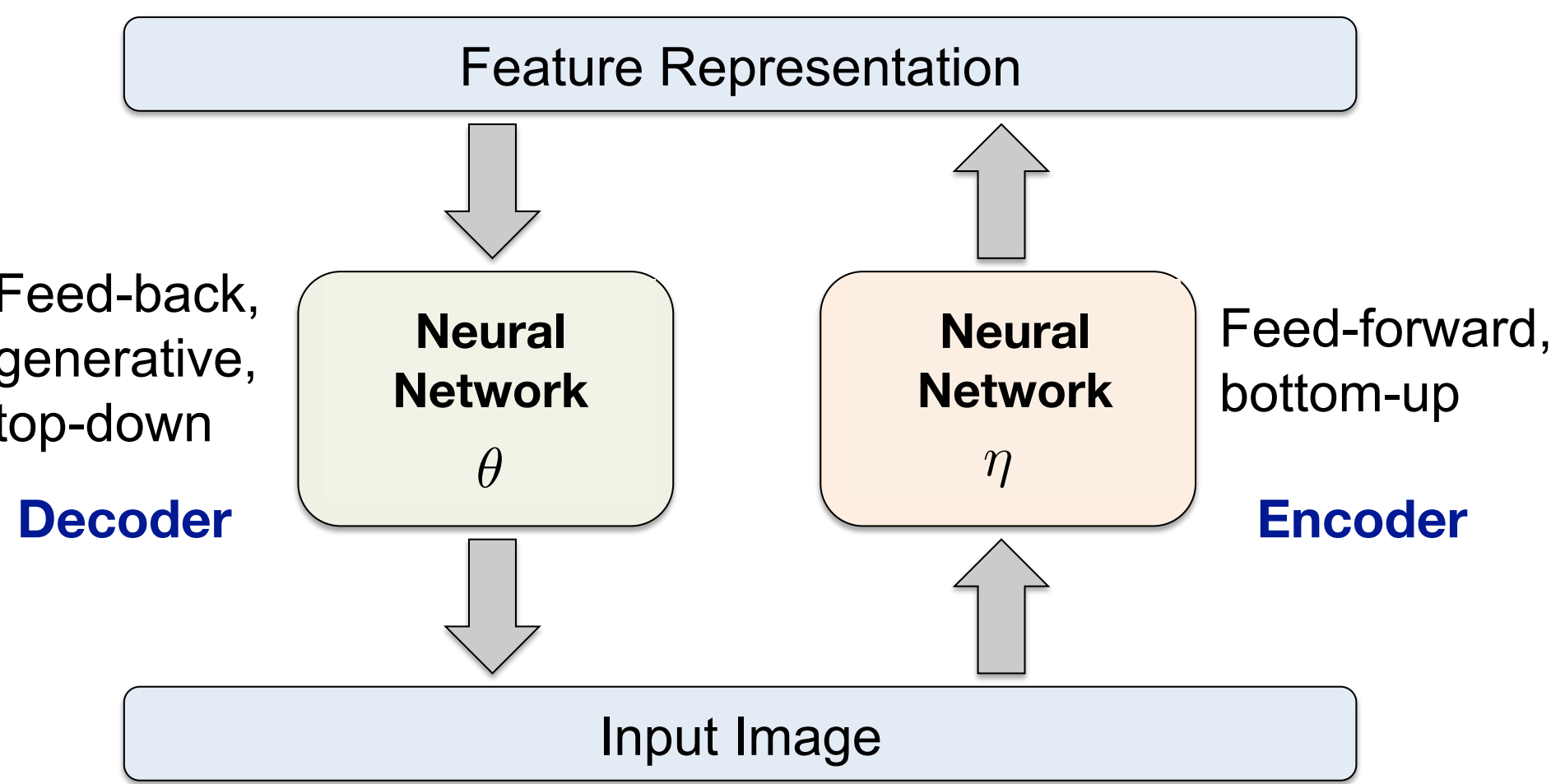
Deep Autoencoders



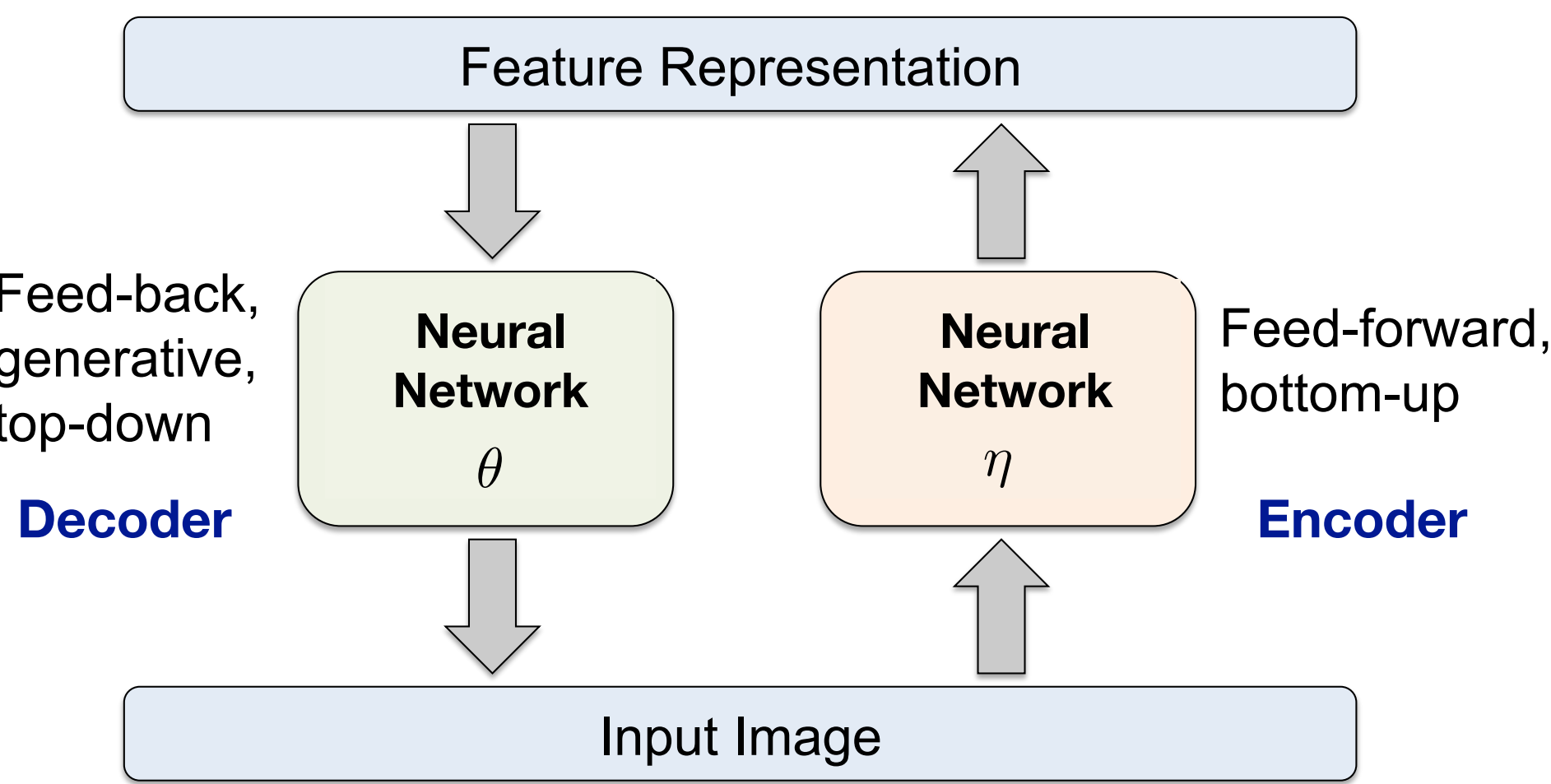
Deep Autoencoders



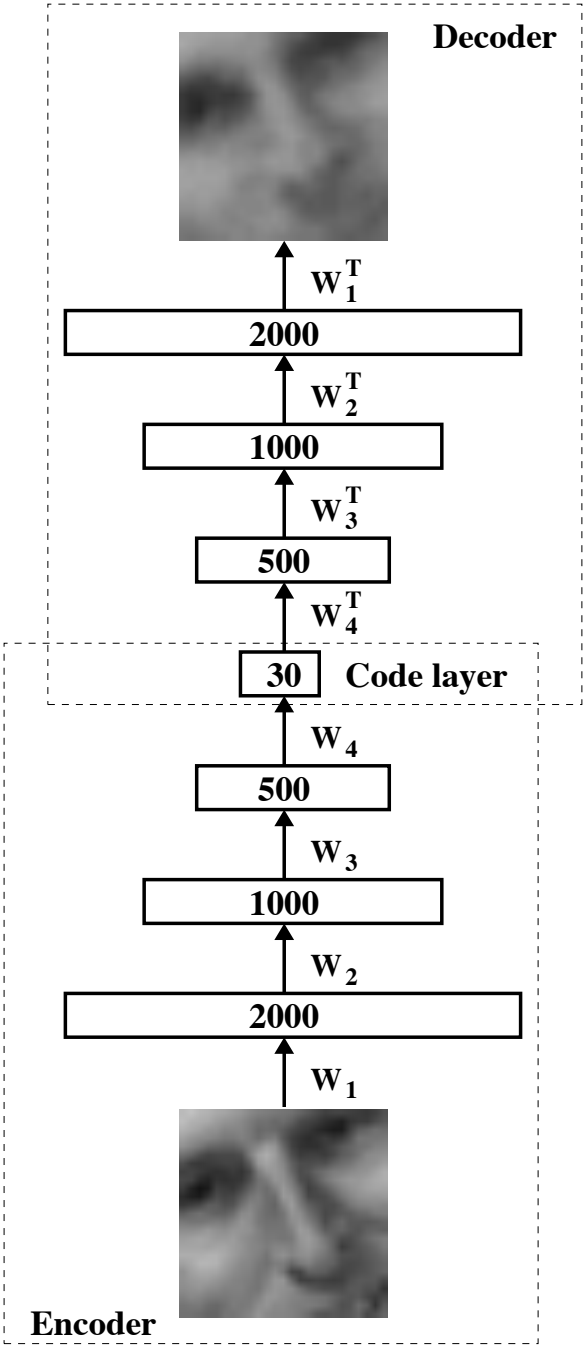
Deep Autoencoders



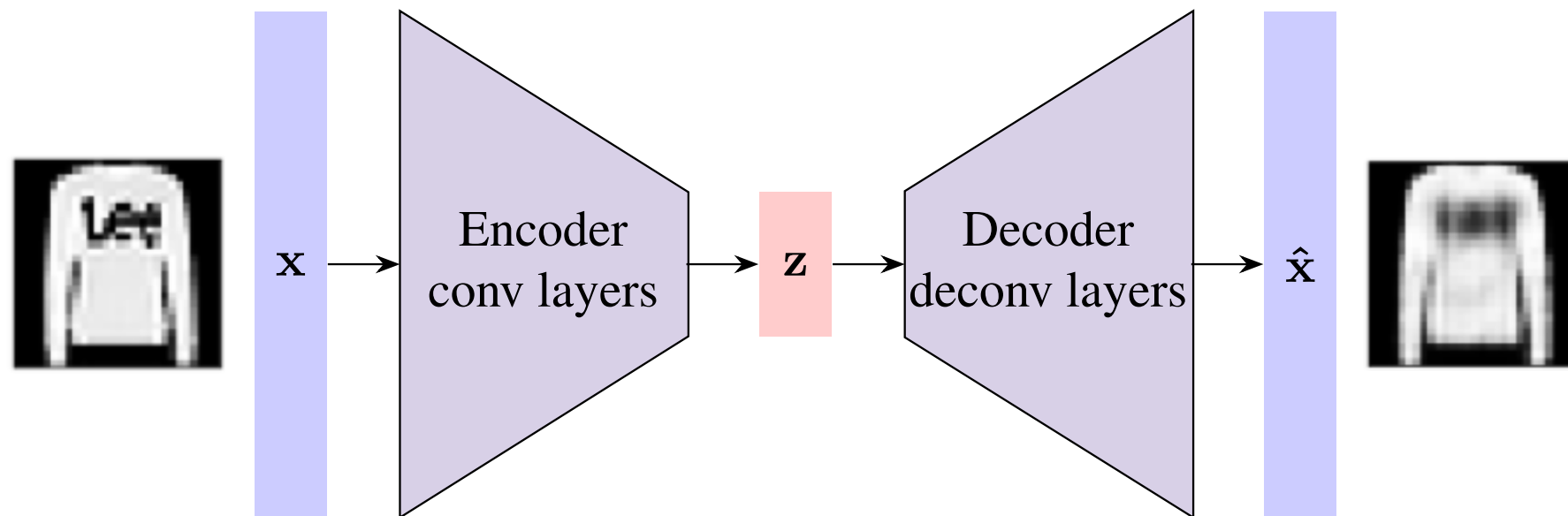
Deep Autoencoders



$$\mathcal{L}(\eta, \theta) = \frac{1}{N} \sum_{n=1}^N \mathcal{L}_n (\mathbf{x}_n - D_{\theta} (E_{\eta}(\mathbf{x}_n)))$$



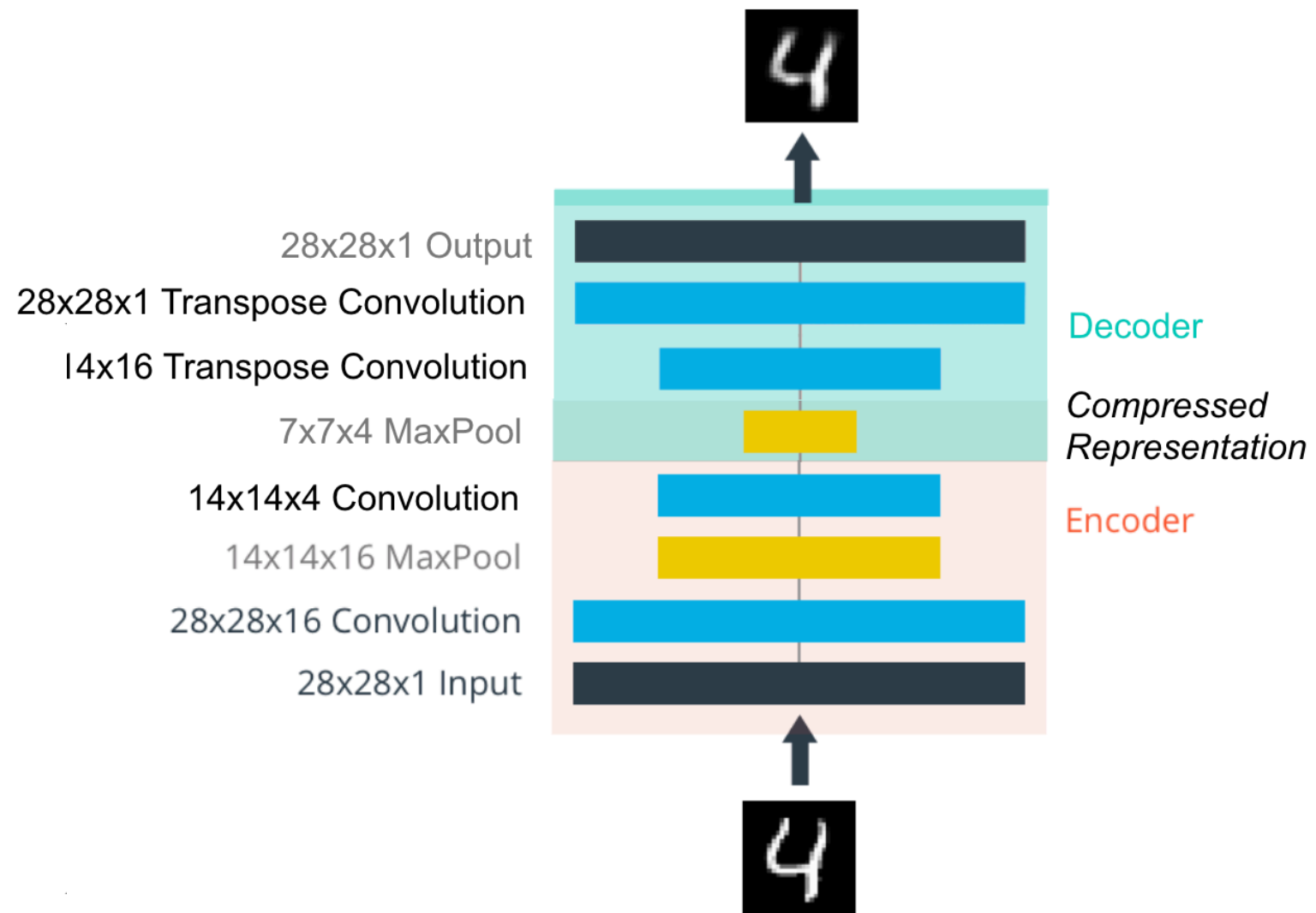
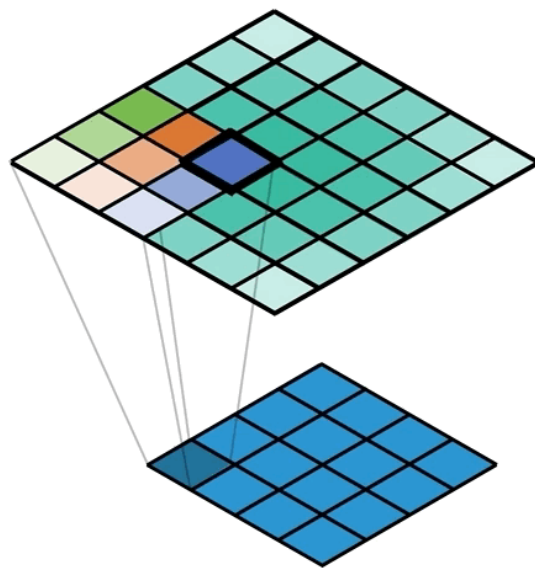
Convolutional Autoencoders



- Considered convolutional autoencoder if one or more (de)-convolutional layers

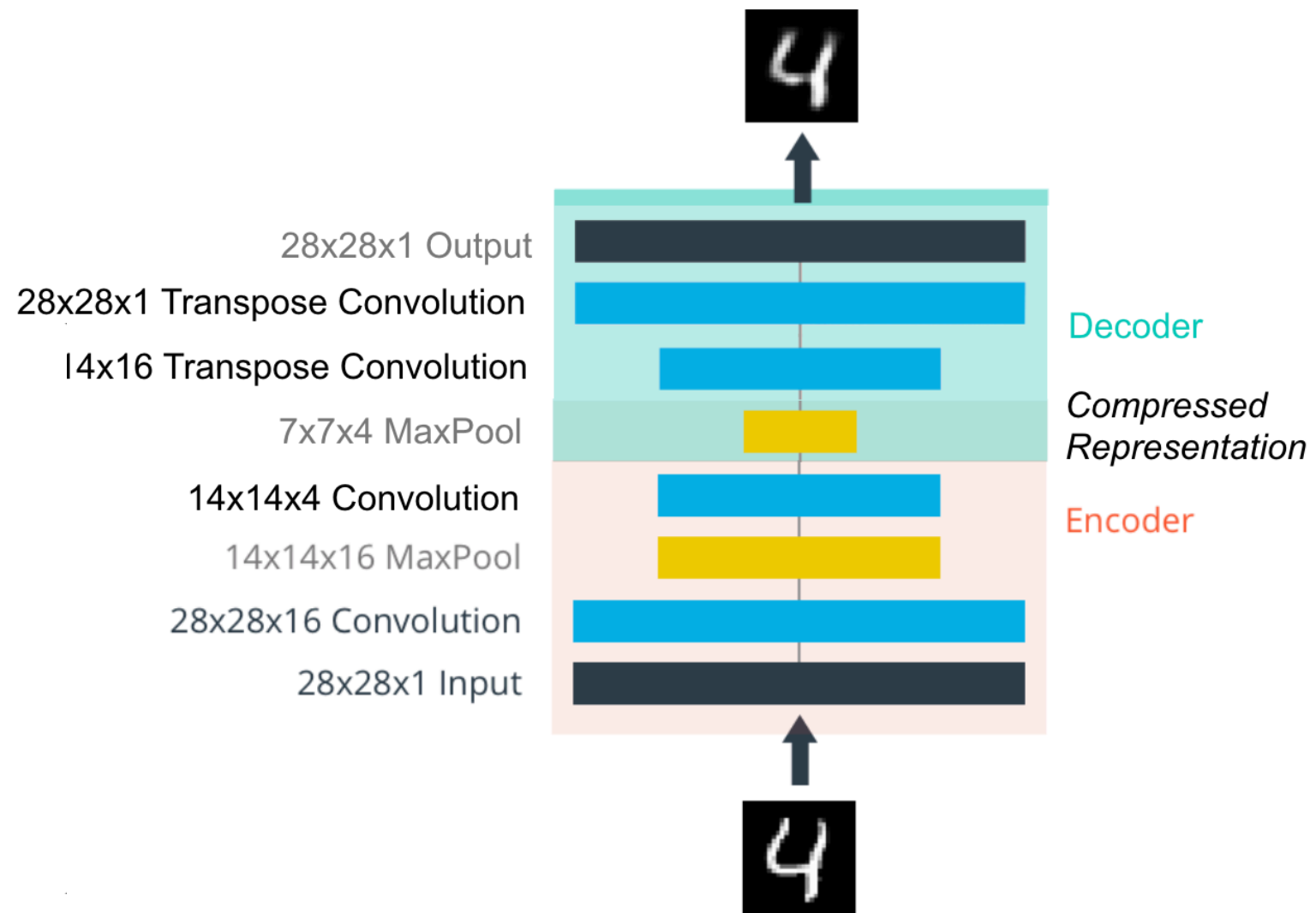
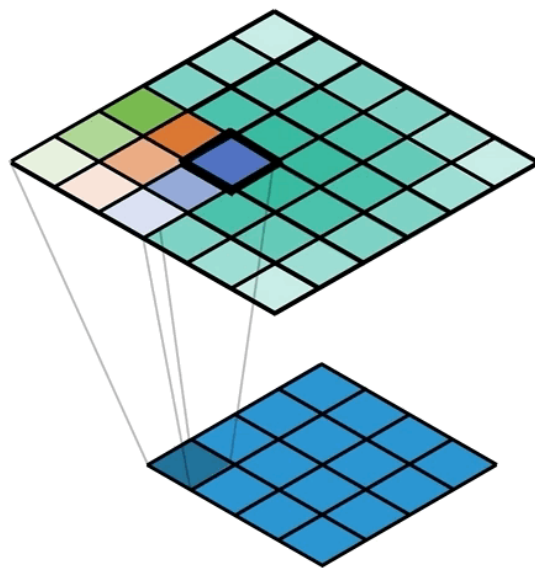
Autoencoders for images. Transpose convolution layers

- The transposed convolution operation forms the same connectivity as the normal convolution but in the **backward direction**
- We can use it to **conduct up-sampling**
- The **weights in the transposed convolution are learnable**. So we do not need a predefined interpolation method.



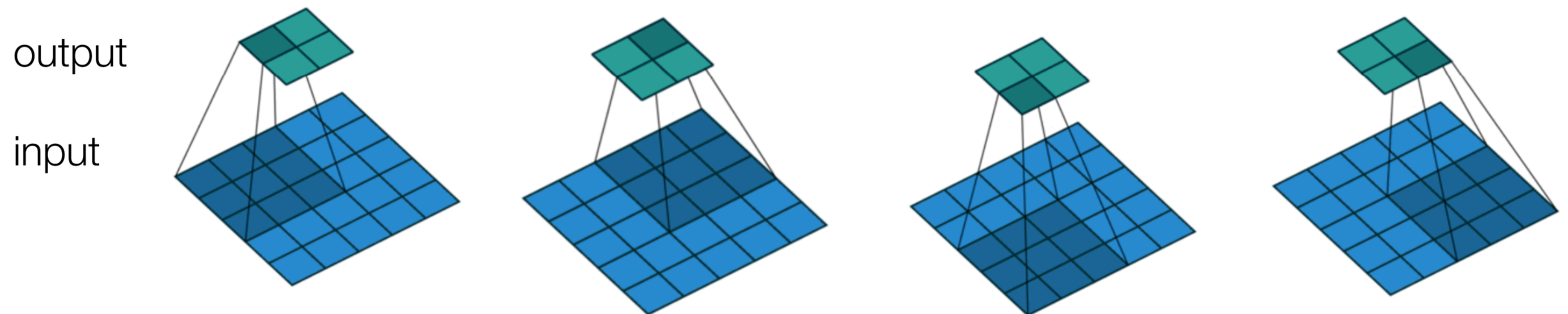
Autoencoders for images. Transpose convolution layers

- The transposed convolution operation forms the same connectivity as the normal convolution but in the **backward direction**
- We can use it to **conduct up-sampling**
- The **weights in the transposed convolution are learnable**. So we do not need a predefined interpolation method.



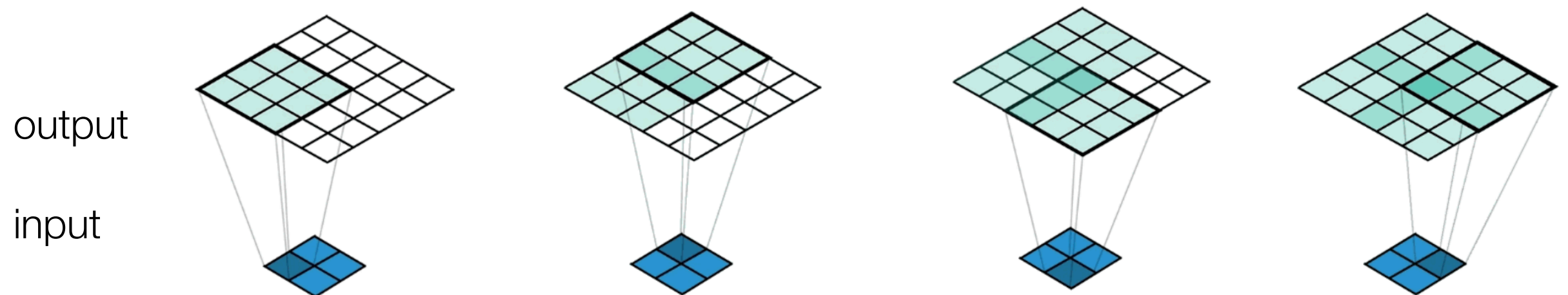
Comparison between convolution and transposed convolution

- Regular convolution:



Dumoulin, Vincent, and Francesco Visin. "A guide to convolution arithmetic for deep learning." [arXiv preprint arXiv:1603.07285](https://arxiv.org/abs/1603.07285) (2016).

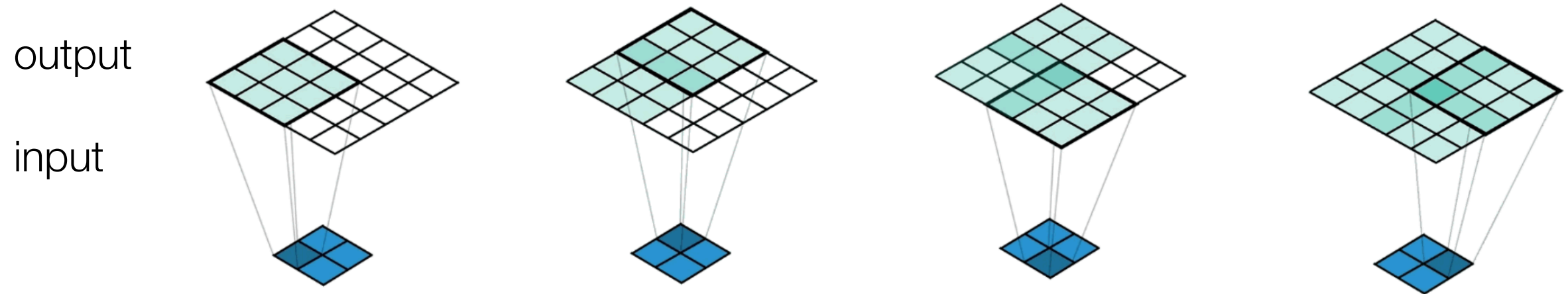
- Transposed convolution (stride 2)



A Conv2DTranspose with 3x3 kernel and stride of 2x2 applied to a 2x2 input to give a 5x5 output.
(<https://medium.com/apache-mxnet/transposed-convolutions-explained-with-ms-excel-52d13030c7e8>)

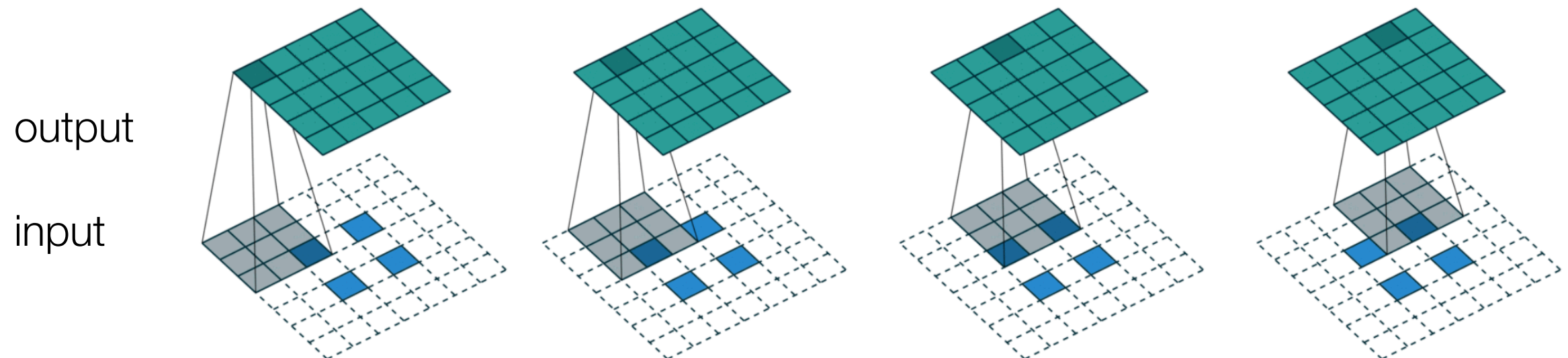
Comparison between convolution and transposed convolution

- Transposed Convolution (3x3 kernel, stride=2):



A Conv2DTranspose with 3x3 kernel and stride of 2x2 applied to a 2x2 input to give a 5x5 output.
(<https://medium.com/apache-mxnet/transposed-convolutions-explained-with-ms-excel-52d13030c7e8>)

- Transposed convolution (emulated with direct convolution)



Comparison between convolution and transposed convolution

- Regular convolution (stride 1):

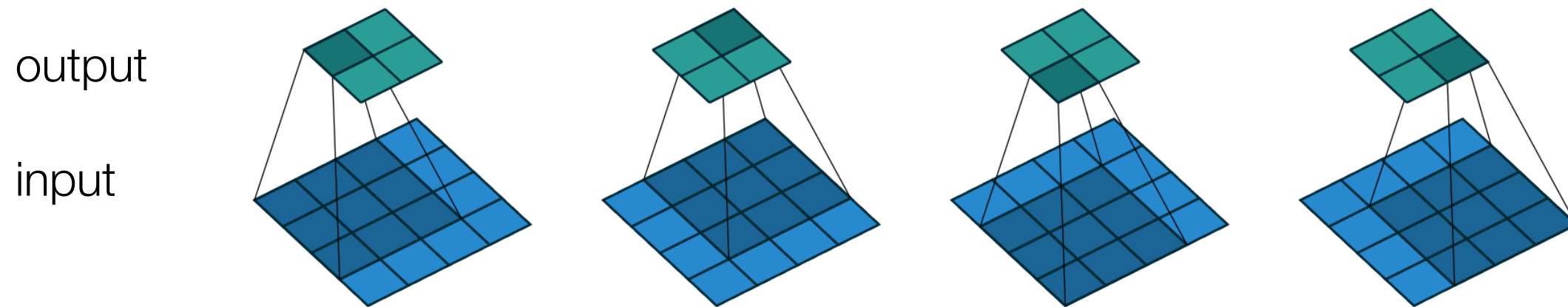
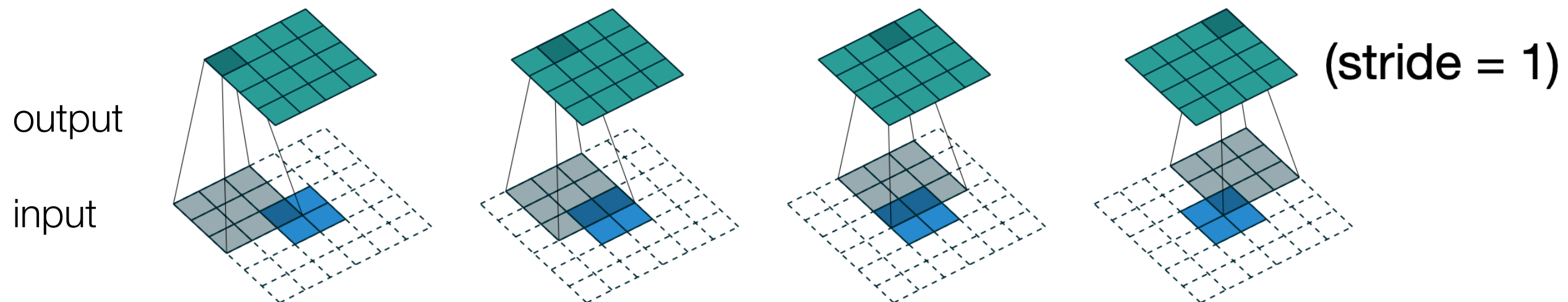


Figure 2.1: (No padding, unit strides) Convolving a 3×3 kernel over a 4×4 input using unit strides (i.e., $i = 4$, $k = 3$, $s = 1$ and $p = 0$).

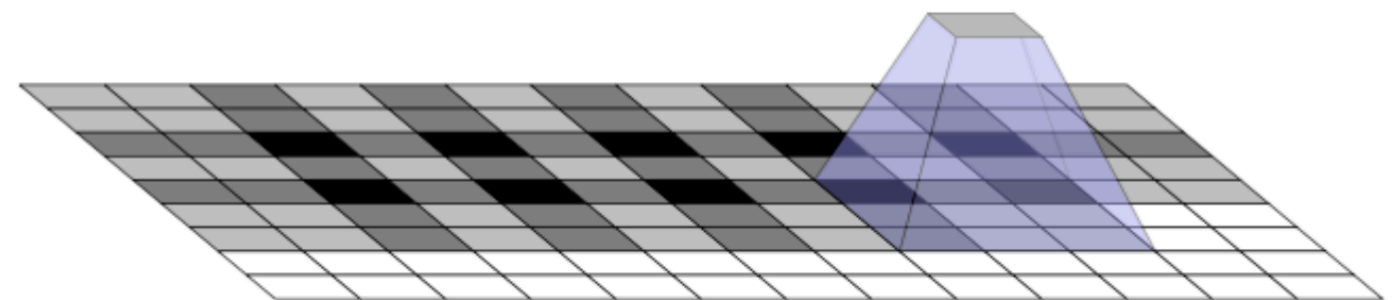
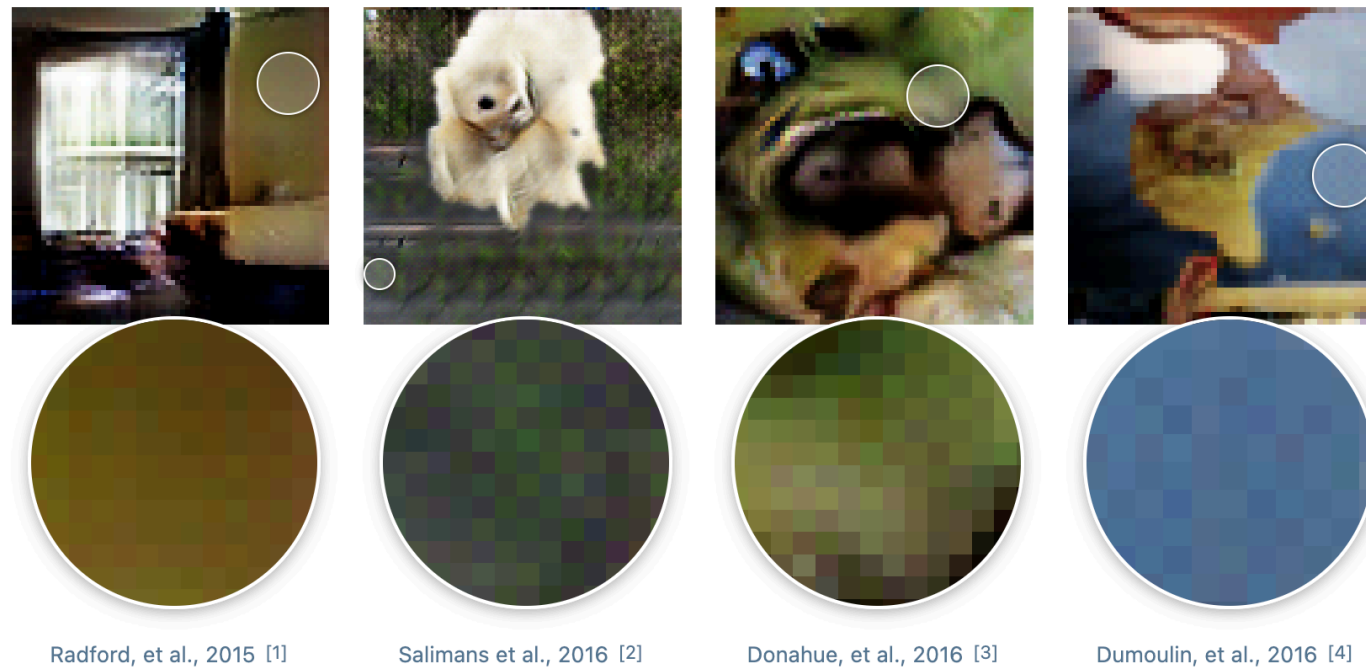
- Transposed convolution (emulated with the direct convolution)



Deconvolution and checkerboard artifacts

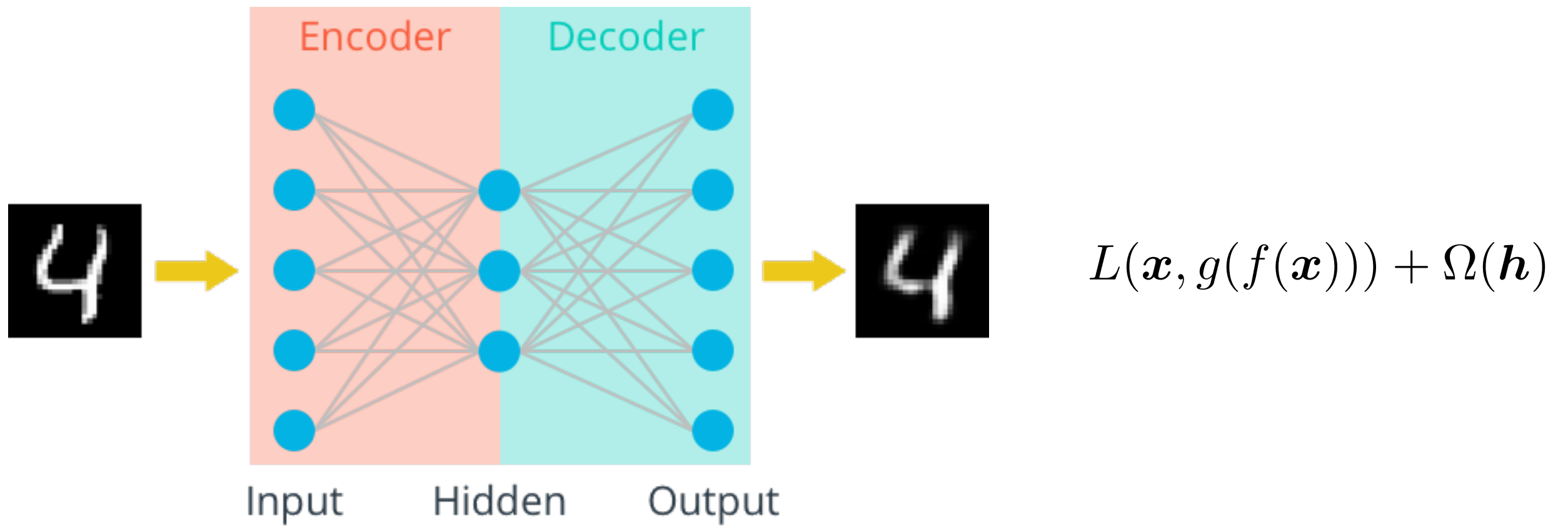
When we look very closely at images generated by neural networks, we often see a strange checkerboard pattern of artifacts. It's more obvious in some cases than others, but a large fraction of recent models exhibit this behavior.

[Link](#)



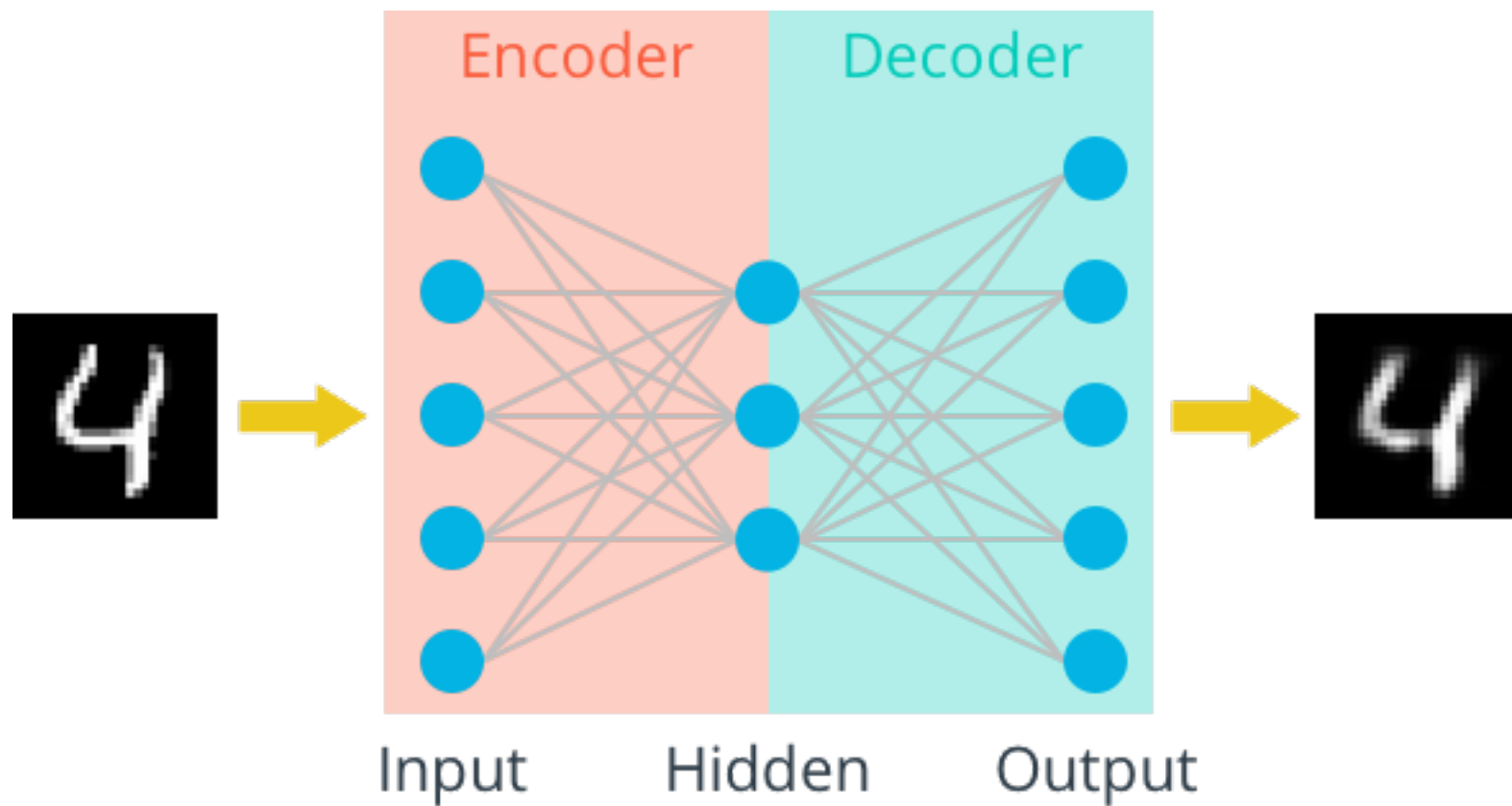
- **Recommendation:** Replace transposed convolution by **upsampling interpolation + regular convolution**

Sparse Deep Autoencoders



- Sparse autoencoders are typically used to learn features for another task

Sparse Deep Autoencoders



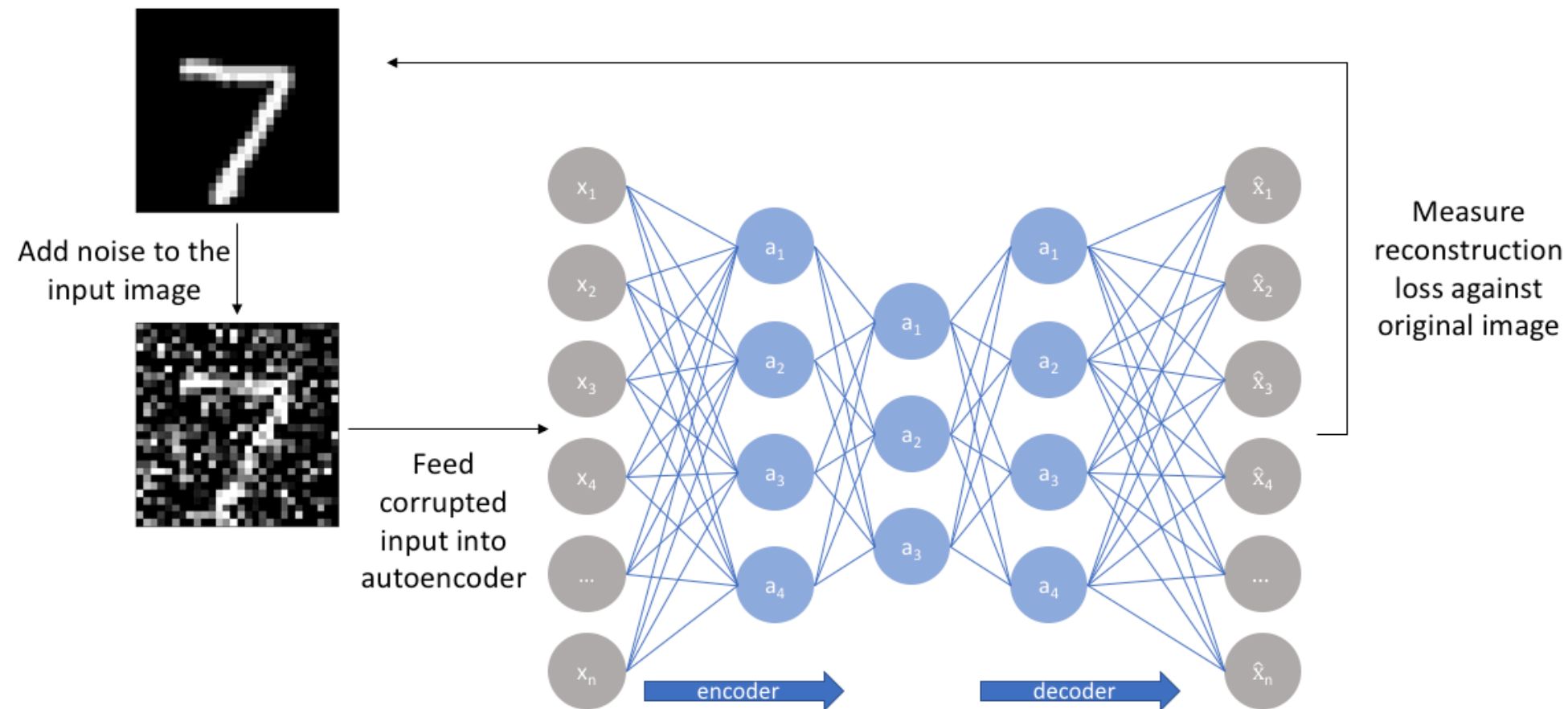
$$L(\mathbf{x}, g(f(\mathbf{x}))) + \Omega(\mathbf{h})$$

Sparsity penalizer

$$\Omega(\mathbf{h}) = \lambda \sum_i |h_i|$$

- Sparse autoencoders are typically used to learn features for another task

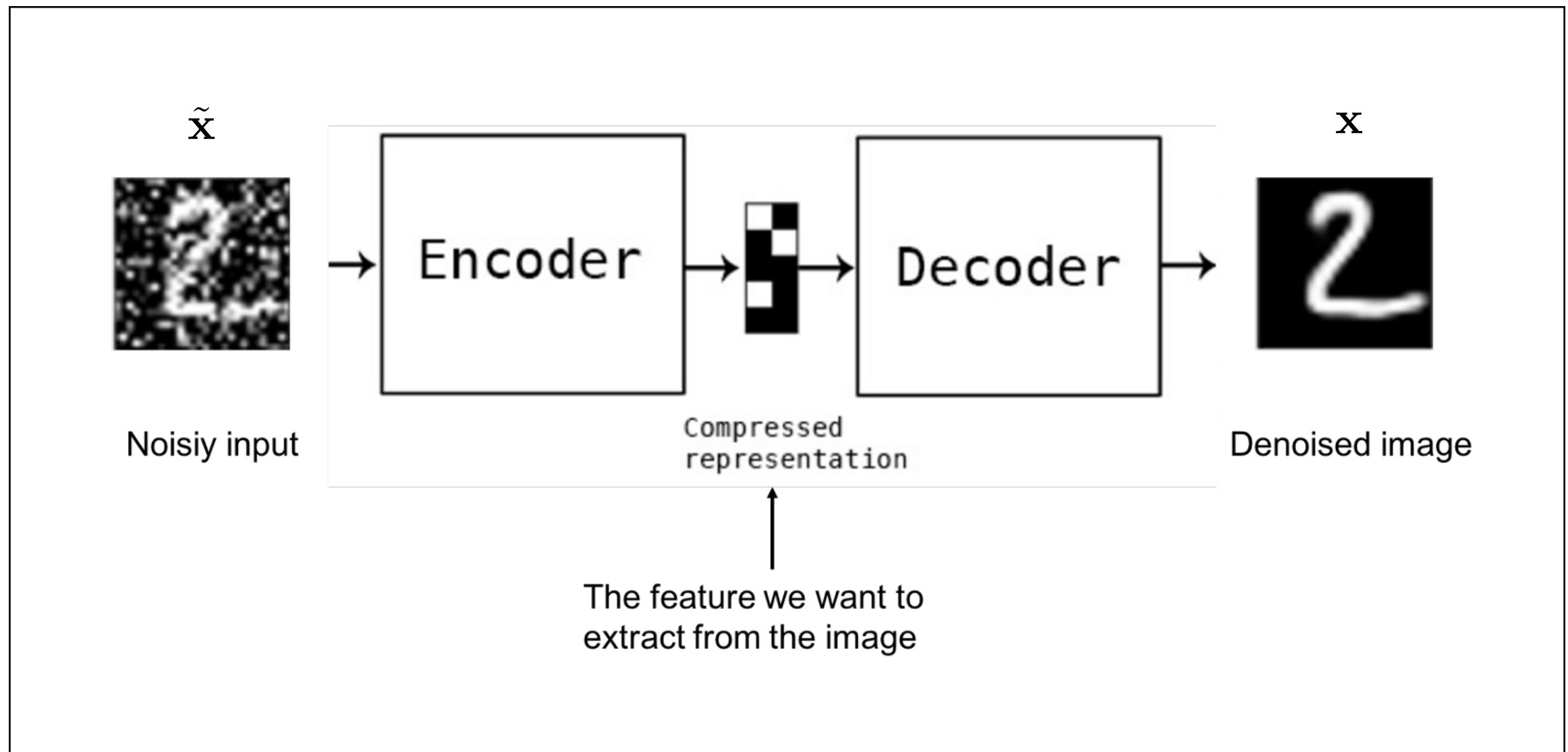
Denoising Deep Autoencoders



Source: [this excellent blog](#)

- Reconstruct a **corrupted version** of an image
- More robust solutions. It is some sort of regularization
- They are widely used for image denoising and missing data completion

Denoising Deep Autoencoders

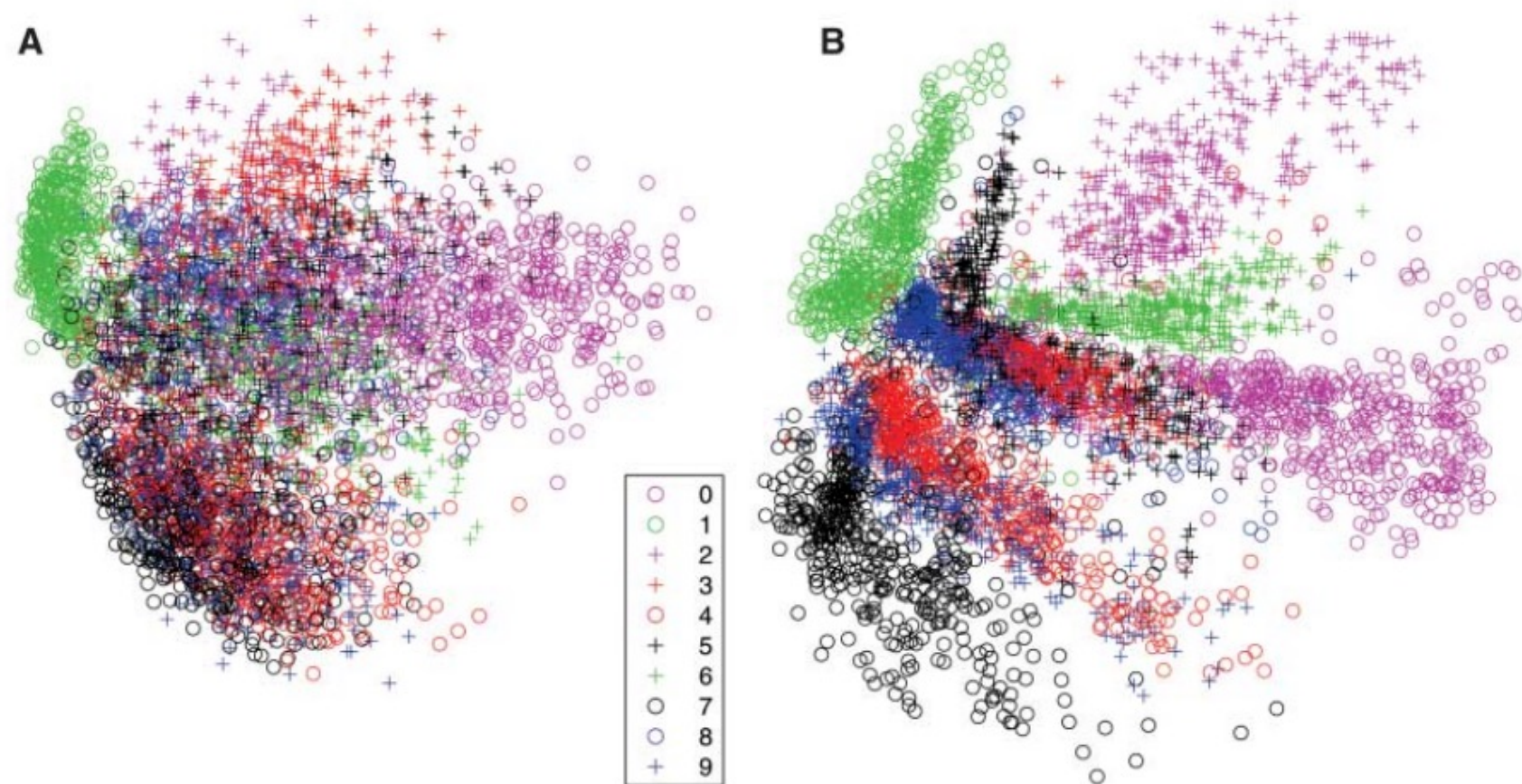


$$\mathcal{L}(\eta, \theta) = \frac{1}{N} \sum_{n=1}^N \mathcal{L}_n(\mathbf{x}_n - D_{\theta}(E_{\eta}(\tilde{\mathbf{x}}_n)))$$

Applications

- Dimensionality reduction
- Visualization

Fig. 3. (A) The two-dimensional codes for 500 digits of each class produced by taking the first two principal components of all 60,000 training images. (B) The two-dimensional codes found by a 784-1000-500-250-2 autoencoder. For an alternative visualization, see (8).



“Reducing the Dimensionality of Data with Neural Networks”, G. E. Hinton and R. R. Salakhutdinov, Science, vol. 313, 2006.

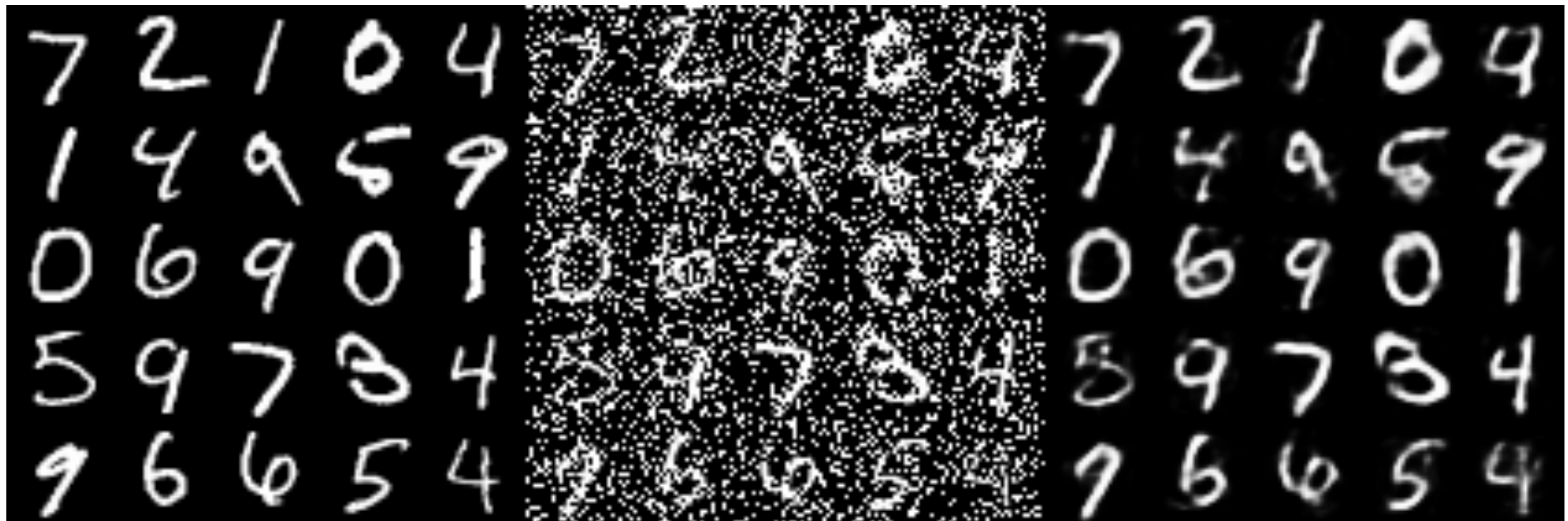
Applications

- Denoising
- Missing data imputation

Original

Corrupted (noisy)

Reconstructed (denoised)



Applications

- Anomaly/outlier detection

**95% of Driving Data:
(1) sunny, (2) highway, (3) straight road**



Edge Cases



Harsh Weather



Pedestrians

Image segmentation using CNNs

Image Segmentation Using Deep Learning: A Survey

Shervin Minaee, Yuri Boykov, Fatih Porikli, Antonio Plaza, Nasser Kehtarnavaz, and Demetri Terzopoulos

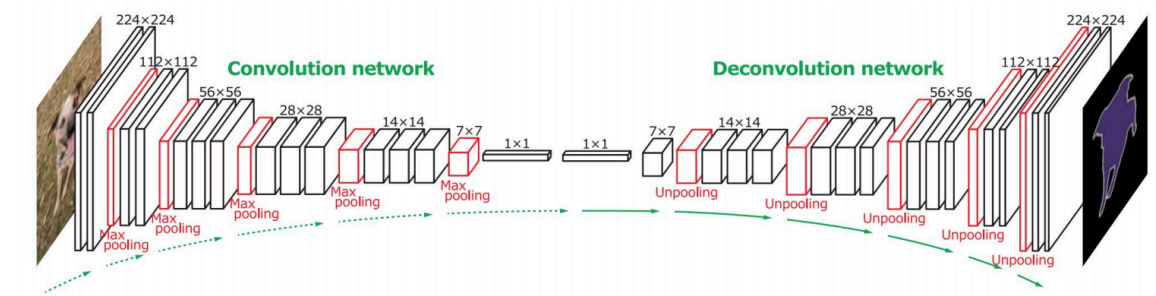


Fig. 11. Deconvolutional semantic segmentation. Following a convolution network based on the VGG 16-layer net, is a multi-layer deconvolution network to generate the accurate segmentation map. From [42].

- Most powerful methods are based on **encoder-decoder networks**

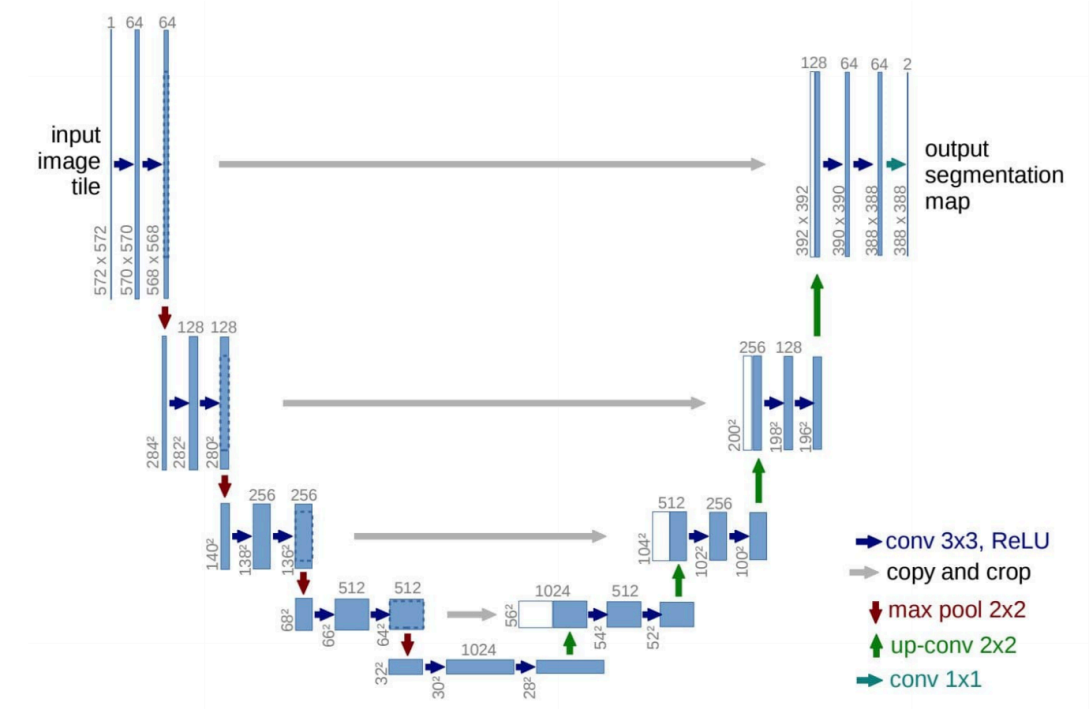


Fig. 14. The U-net model. The blue boxes denote feature map blocks with their indicated shapes. From [49].

Image segmentation using CNNs

- Image segmentation using CNNs

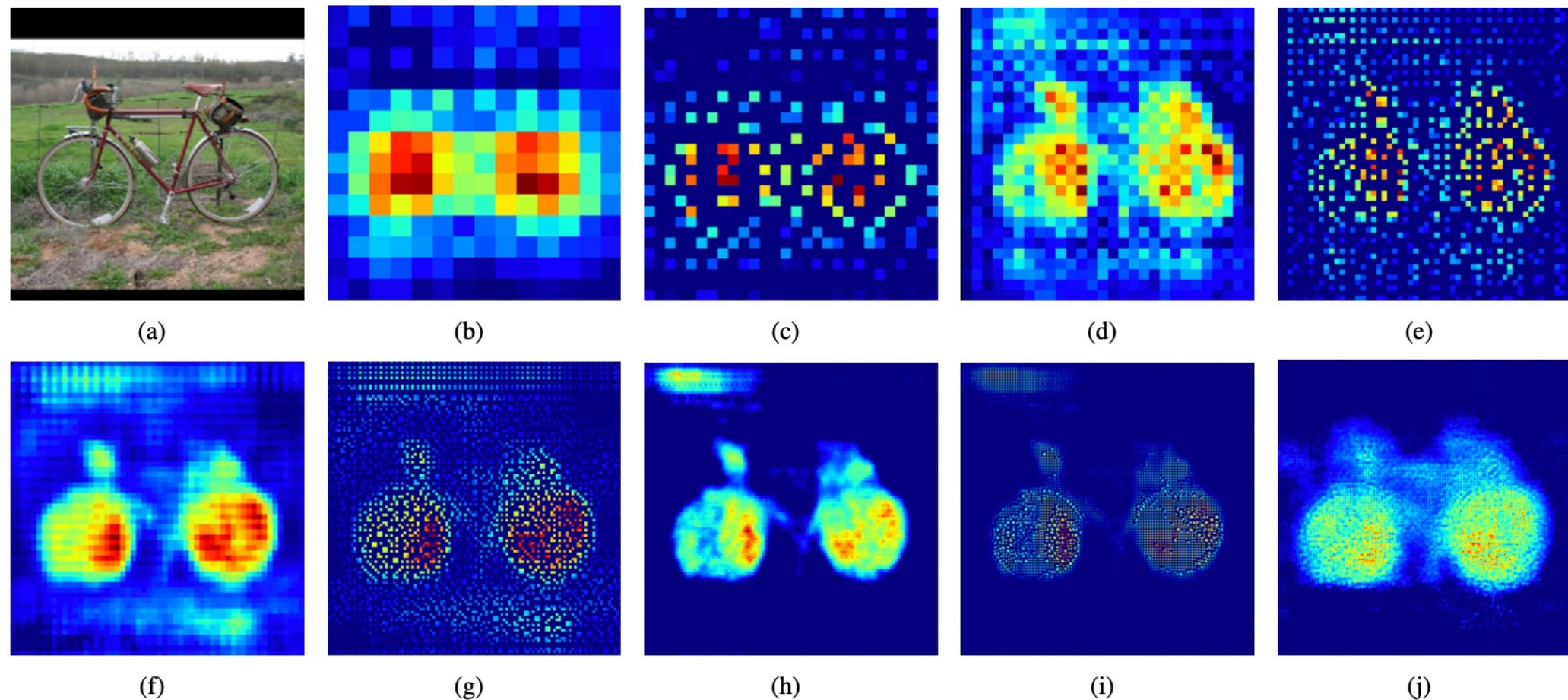
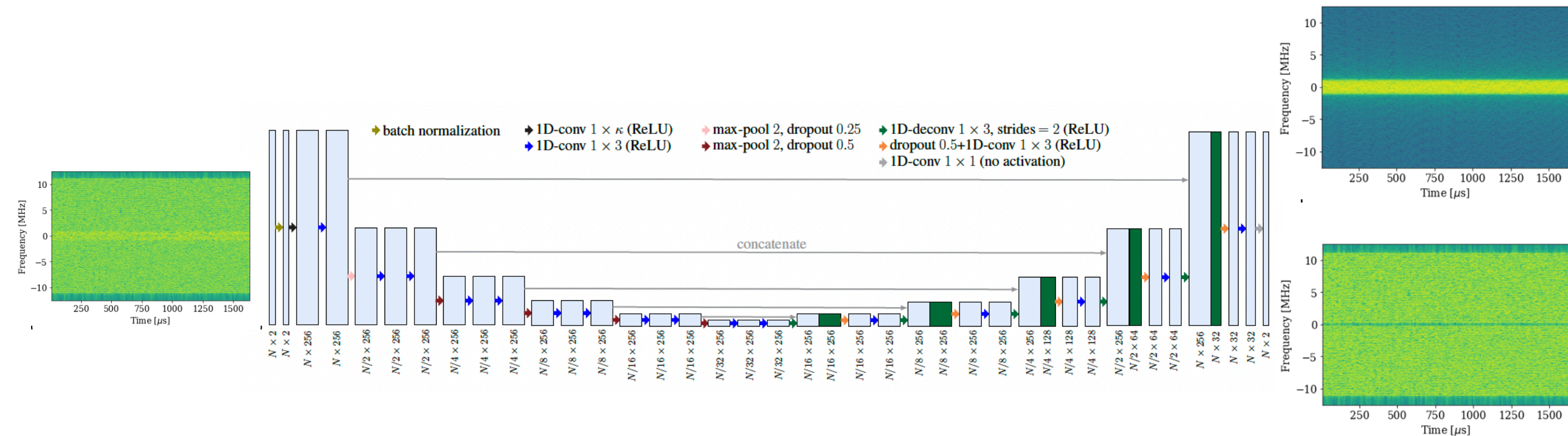


Figure 4. Visualization of activations in our deconvolution network. The activation maps from top left to bottom right correspond to the output maps from lower to higher layers in the deconvolution network. We select the most representative activation in each layer for effective visualization. The image in (a) is an input, and the rest are the outputs from (b) the last 14×14 deconvolutional layer, (c) the 28×28 unpooling layer, (d) the last 28×28 deconvolutional layer, (e) the 56×56 unpooling layer, (f) the last 56×56 deconvolutional layer, (g) the 112×112 unpooling layer, (h) the last 112×112 deconvolutional layer, (i) the 224×224 unpooling layer and (j) the last 224×224 deconvolutional layer. The finer details of the object are revealed, as the features are forward-propagated through the layers in the deconvolution network. Note that noisy activations from background are suppressed through propagation while the activations closely related to the target classes are amplified. It shows that the learned filters in higher deconvolutional layers tend to capture class-specific shape information.

Applications: UNet for Source Separation of Radio Frequency Signals



- **Idea:** Use the **UNet autoencoder** structure as an **advanced denoiser**
- **Source separation** of two components equivalent to denoising
- Here data is quite different from images, but a **properly designed UNet can also work**

[Video demo link here](#)